Встреча 6

Разработка микросервисных приложений на Golang

YADRO

Экватор пройден!



Часто встречающиеся ошибки и замечания

- почти нет!

Интересные моменты

- Поиск через Go vs Postgres (maps vs SQL)
- Ранжирование: l слов - фраза, m слов - комикс, n слов - пересечение
  - order by n
  - order by n / l
  - order by n / m
  - order by n / sqrt(m*m + n*n)
  - …
- BM25 👀

SQL vs Go

Подходы

- поле []TEXT и unnest() vs несколько нормализованных таблиц
- = ANY($1::TEXT[]) vs && $1::text[] vs INTERSECT vs IN
- cardinality(words) vs array_length(words, 1)
- несколько запросов vs подзапросы всех видов, включая CTE vs JOIN всех видов, включая LATERAL

1 Статистика

1.1. Простые запросы (зато два) и логика на Go:

```
SELECT COUNT(*) FROM comics;

SELECT words FROM comics;

#+Go code
```

1 Статистика

1.2. Три запроса в БД (три отдельных транзакции и скорее всего три коннекта):

```
SELECT COUNT(*) FROM comic_words;

SELECT COUNT(DISTINCT word) FROM comic_words;

SELECT COUNT(*) FROM comics;
```

1 Статистика

1.3. Один запрос. Но всё равно три:

```
SELECT
(SELECT COUNT(*) FROM comics) AS comics_fetched,
(SELECT COUNT(DISTINCT word) FROM words) AS words_unique,
(SELECT COUNT(*) FROM words) AS words_total;
```

1 Статистика

1.4. Мощь Постгреса с JOIN LATERAL (но array_length):

```
SELECT
    COUNT(DISTINCT c.id) AS comics_fetched,
    COALESCE(SUM(array_length(c.words, 1)), 0) AS words_total,
    COUNT(DISTINCT w.word) AS words_unique
FROM comics c
LEFT JOIN LATERAL unnest(c.words) AS w(word) ON TRUE
```

1 Статистика

1.5. Мощь Постгреса с CTE (но два прохода по таблице):

```
WITH all_words AS (
    SELECT unnest(words) AS word
    FROM comics
)
SELECT
    COALESCE(SUM(cardinality(words)), 0) AS words_total,
    COALESCE((SELECT COUNT(DISTINCT word) FROM all_words), 0) AS words_unique,
    COUNT(*) AS comics_fetched
FROM comics;
```

2 Поиск

2.1. Тривиальный запрос и логика на Go:

SELECT id, url, words FROM comics;

#+Go code

2 Поиск

2.2. Простой запрос с фильтрацией и логика ранжирования на Go (ANY vs &&):

```
SELECT id, img_url
FROM comics
WHERE $1 = ANY(words)
# + Go-code for ranking

SELECT id, url, keywords FROM comics WHERE keywords && $1
#+Go code for ranking
```

2 Поиск

2.3. Расчёт релевантности средствами СУБД в нормализованной базе:

```
SELECT c.id, c.url, COUNT(DISTINCT cw.word) as score
FROM comics c
JOIN comic_words cw ON c.id = cw.comic_id
WHERE cw.word IN (%s)
GROUP BY c.id, c.url
ORDER BY score DESC, c.id ASC
LIMIT $%d
```

2 Поиск

2.4. Расчёт релевантности средствами СУБД в массиве, с группировкой:

```
SELECT id, url
FROM comics, unnest(words) AS word
WHERE word = ANY($1)
GROUP BY id, url
ORDER BY count(word) DESC
LIMIT $2;
```

2 Поиск

2.5. Расчёт релевантности средствами СУБД в массиве, с подзапросом:

```
SELECT id, url
FROM comics
WHERE words && $1::text[]
ORDER BY array_length( array (
        SELECT unnest(words) INTERSECT SELECT unnest($1::text[])), 1) DESC,
    array_length(words, 1) ASC
LIMIT $2
```


22:00
Давай еще паттернов?
Абстрактную фабрику?
Ага!
5:30